

UNIVERSIDAD SERGIO ARBOLEDA

Sistema inteligente para planificación de rutas de taxis en una zona de estudio de Chapinero.

Primera entrega: agente inteligente y formulación

Castañeda Lina

García Jorge

Hernandez Brayan

Inteligencia Artificial

Joaquín Sánchez

Ciencias de la Computación e Inteligencia Artificial

Septiembre de 2026

1. Introducción

La planificación de rutas urbanas no depende únicamente de la distancia. En una red vial también influyen el sentido de circulación, el tiempo estimado, la velocidad, la congestión, los incidentes y la disponibilidad de cada segmento. Por esta razón, el problema se aborda mediante un agente inteligente que percibe el estado de la red, evalúa acciones y avanza hacia un destino según un criterio definido.

El caso de aplicación se enfoca en taxis que operan en una zona de estudio de Chapinero. Esta delimitación permite trabajar con datos viales reales y conservar un tamaño manejable para implementar, probar y comparar los algoritmos exigidos por la asignatura.

2. Planteamiento del problema

Un taxi debe desplazarse entre puntos de una red vial respetando calles de uno o doble sentido. La ruta más corta en distancia no siempre coincide con la ruta de menor tiempo, y una vía aparentemente conveniente puede presentar congestión, incidentes o restricciones. Se requiere un sistema capaz de representar estas condiciones, calcular alternativas y explicar la recomendación al usuario.

2.1 Delimitación

- La zona de estudio corresponde a la red transitable por automóvil en un radio aproximado de 1.200 m alrededor de la Universidad Sergio Arboleda.
- El prototipo actual resuelve el recorrido entre dos puntos seleccionados: origen y destino.
- La congestión e incidentes están modelados, pero los datos actuales son valores iniciales o simulados.

3. Objetivos

3.1 Objetivo general

Desarrollar un sistema inteligente para representar y planificar rutas de taxi en una zona de estudio de Chapinero mediante grafos dirigidos, algoritmos de búsqueda y, en fases posteriores, modelos de aprendizaje automático.

3.2 Objetivos específicos

1. Construir una red vial dirigida a partir de datos reales de OpenStreetMap.
2. Modelar intersecciones y segmentos viales con atributos de distancia, tiempo, velocidad, congestión y disponibilidad.
3. Diseñar un agente basado en objetivos y utilidad mediante el modelo PEAS.
4. Implementar y comparar BFS, DFS, UCS, búsqueda voraz y A*.
5. Construir un modelo de regresión para estimar el tiempo de desplazamiento.

4. Alcance funcional

- El proyecto abarcará una zona de estudio de Chapinero, Bogotá, delimitada por un radio aproximado de 1.200 metros alrededor de la Universidad Sergio Arboleda. Se utilizarán las vías transitables por automóvil registradas en OpenStreetMap.
- La red vial se representará mediante un grafo dirigido. Los nodos corresponderán a intersecciones o puntos de conexión, mientras que las aristas representarán segmentos viales y conservarán los sentidos de circulación registrados.
- El sistema permitirá seleccionar un punto de origen, un destino y un criterio de optimización para planificar el recorrido de un taxi dentro de la zona estudiada.
- Las conexiones incluirán atributos como distancia, tiempo estimado, velocidad, congestión y disponibilidad. El agente solo podrá desplazarse por conexiones habilitadas en el modelo.
- Se implementarán y compararán los algoritmos BFS, DFS, UCS, búsqueda voraz y A*. Los recorridos y sus métricas se mostrarán mediante una interfaz con visualización del mapa y del grafo.
- Posteriormente se incorporarán modelos de aprendizaje automático para predecir tiempos de desplazamiento, clasificar niveles de congestión y agrupar zonas con características similares.
- Inicialmente, las condiciones de congestión e incidentes se representarán mediante valores definidos o simulados. El sistema no contará con seguimiento de taxis ni información de tráfico en tiempo real.

4.1 Herramientas y tecnologías utilizadas en el primer corte

Herramienta	Uso en este corte
Python	Desarrollo del grafo y del prototipo del agente.
OpenStreetMap	Fuente de los datos de calles e intersecciones de la zona de estudio.
OSMnx	Descarga y procesamiento de la red vial.
NetworkX	Representación y consulta de los nodos y conexiones del grafo.

Pydantic	Validación de los datos de nodos, aristas y solicitudes de la API.
FastAPI	Creación de la API para consultar el grafo, sus estadísticas y ejecutar pasos del agente.
Nuxt y Vue	Construcción de la interfaz para seleccionar origen y destino y consultar la información.
Leaflet	Visualización geográfica de la red vial sobre un mapa.
Cytoscape.js	Visualización interactiva de los nodos y las conexiones del grafo.
Pytest	Definición de pruebas unitarias del grafo, del modelo PEAS y del motor de decisión.

5. Diseño del agente inteligente

El agente recibe un origen, un destino, una fecha, una hora y un criterio de optimización. Después consulta las conexiones salientes disponibles desde su posición, selecciona una acción y actualiza su estado.

5.1 Modelo PEAS

Para describir el funcionamiento del agente se utiliza el modelo PEAS, compuesto por la medida de desempeño, el ambiente, los actuadores y los sensores.

Componente	Aplicación en el proyecto
P - Medida de desempeño	Alcanzar el destino con una ruta válida; minimizar tiempo, distancia o congestión; evitar ciclos, segmentos bloqueados e incidentes.
E - Ambiente	Red vial dirigida, intersecciones, calles, sentidos, velocidades, congestión, incidentes, fecha y hora.
A - Actuadores	Avanzar al siguiente nodo, esperar, recalcular y finalizar el recorrido.

S - Sensores	Ubicación actual, origen, destino, hora, conexiones disponibles, distancia, tiempo, congestión e incidentes.
--------------	--

5.2 Medida de desempeño

El desempeño del agente se evaluará según su capacidad para alcanzar el destino mediante un recorrido válido dentro de la red vial. Se tendrán en cuenta los siguientes criterios:

- Alcanzar el punto de destino seleccionado.
- Utilizar conexiones existentes y habilitadas en el grafo.
- Respetar los sentidos de circulación representados.
- Evitar ciclos y desplazamientos innecesarios.
- Registrar el tiempo estimado y la distancia total del recorrido.
- Comparar las rutas según el criterio de optimización seleccionado.

Criterio seleccionado	Medida que debe minimizarse
Menor distancia	Suma de las distancias de los segmentos recorridos.
Menor tiempo estimado	Suma de los tiempos estimados de los segmentos recorridos.
Menor cantidad de conexiones	Número de segmentos viales recorridos.

5.3 Ambiente

El ambiente estará conformado por la red vial de una zona de estudio de Chapinero, Bogotá, delimitada por un radio aproximado de 1.200 metros alrededor de la Universidad Sergio Arboleda. Se utilizarán datos de OpenStreetMap correspondientes a vías transitables por automóvil.

La red se representará mediante un grafo dirigido con 888 nodos y 1.741 aristas. Los nodos corresponderán a intersecciones o puntos de conexión vial, mientras que las aristas representarán los segmentos por los cuales podrá desplazarse el taxi.

El ambiente incluirá:

- Intersecciones y puntos de conexión vial.
- Segmentos de calles que conectan los nodos.
- Sentidos de circulación registrados en la red.
- Distancias de los segmentos.
- Velocidades registradas o estimadas.
- Tiempos estimados de desplazamiento.
- Disponibilidad de las conexiones.
- Atributos de congestión e incidentes.

Características del ambiente

Propiedad	Clasificación	Justificación
Observabilidad	Parcialmente observable	La red estática es conocida, pero el tráfico real no se observa completamente.
Determinismo	Estocástico en operación	El tiempo puede variar aun siguiendo la misma ruta.
Dinamismo	Dinámico	Congestión, incidentes y cierres pueden cambiar.
Espacio	Discreto	El agente decide entre nodos y segmentos definidos.
Temporalidad	Secuencial	Cada movimiento condiciona las acciones siguientes.
Agentes	Multiagente en el mundo real	Existen otros vehículos; el prototipo controla un taxi.
Conocimiento	Parcialmente conocido	La topología se conoce; algunas condiciones deben estimarse.

5.4 Actuadores

Los actuadores permiten ejecutar las acciones seleccionadas por el agente y actualizar su estado dentro de la simulación del recorrido del taxi.

Las acciones previstas son:

- **Avanzar:** desplazarse desde el nodo actual hasta un nodo vecino mediante una conexión habilitada, respetando el sentido de circulación.
- **Esperar:** permanecer en el nodo actual durante un intervalo definido cuando las condiciones del modelo lo requieran.
- **Recalcular:** solicitar una nueva planificación desde la ubicación actual cuando una conexión deje de estar disponible o no sea posible continuar. La búsqueda de una ruta alternativa completa se incorporará en el segundo corte.
- **Finalizar:** terminar el recorrido cuando se alcance el destino seleccionado.
- **Comunicar resultados:** presentar el recorrido realizado, su estado y las métricas disponibles, o informar que el agente no logró continuar.

Estas acciones se ejecutarán dentro del sistema de software. Avanzar representará actualizar la ubicación del taxi en el grafo y registrar el desplazamiento, sin controlar físicamente un vehículo.

5.5 Sensores

Los sensores corresponden a los mecanismos de software mediante los cuales el agente consulta la información proporcionada por el usuario y los datos de la red vial.

Las principales percepciones serán:

- **Punto de origen:** nodo desde el cual comienza el recorrido.
- **Punto de destino:** nodo que el taxi debe alcanzar.
- **Fecha y hora:** datos de referencia para la consulta; inicialmente no modificarán los costos según el tráfico real.
- **Criterio de optimización:** medida seleccionada para evaluar el recorrido.
- **Conexiones disponibles:** segmentos salientes que pueden recorrerse desde el nodo actual, respetando su dirección y disponibilidad.
- **Atributos de las conexiones:** distancia, velocidad y tiempo estimado de desplazamiento.
- **Condiciones del entorno:** valores de congestión, bloqueos e incidentes registrados o simulados en el modelo.

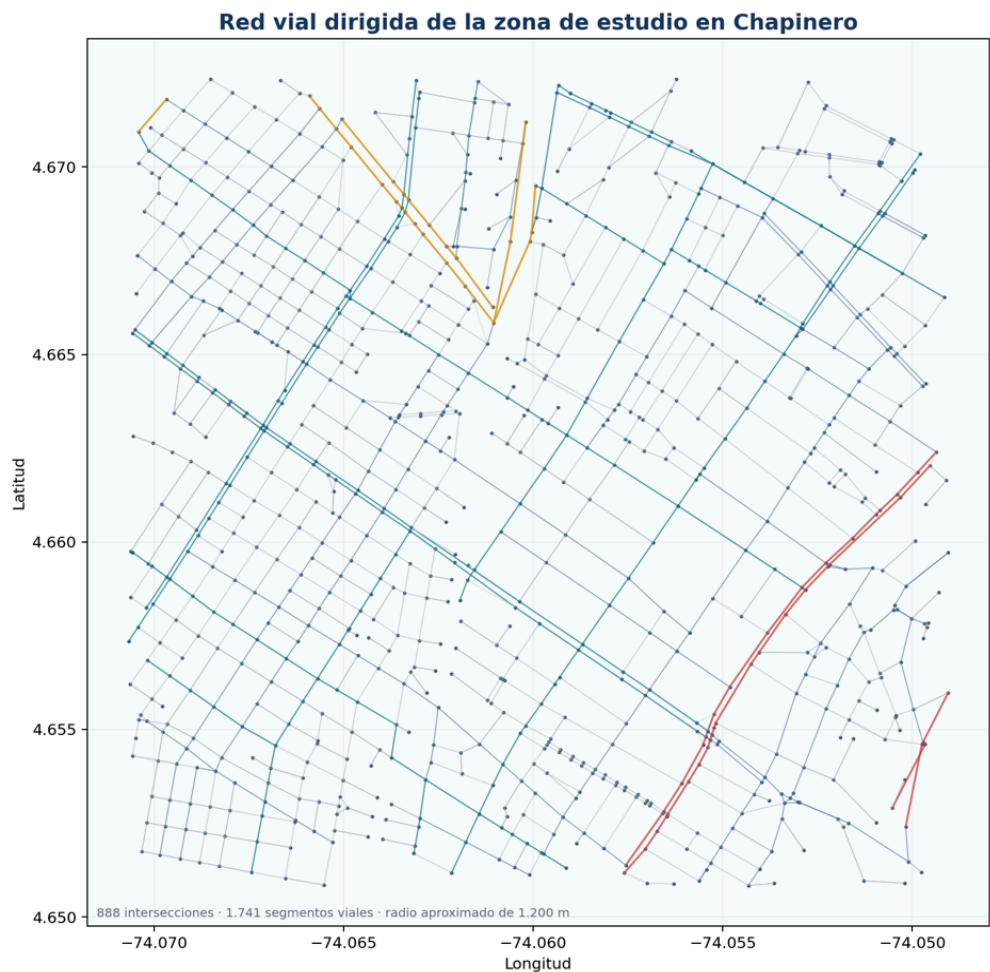
5.6 Tipo de agente

Se modela como un agente basado en objetivos porque busca alcanzar un nodo destino, y con función de utilidad porque puede comparar varias alternativas mediante un costo. En fases posteriores se convertirá en un agente de aprendizaje al incorporar predicción de tiempos y clasificación de congestión.

6. Diagrama de la red

La red se define como un grafo dirigido $G = (V, E)$, donde V es el conjunto de intersecciones y E el conjunto de segmentos viales transitables. La dirección permite conservar el sentido de las calles.

Elemento	Representación
Intersección	Nodo con identificador y coordenadas WGS84.
Segmento vial	Arista dirigida entre dos intersecciones.
Sentido vial	Orientación origen-destino de la arista.
Peso	Distancia o tiempo estimado, según el criterio.
Disponibilidad	Atributo booleano para habilitar o bloquear un segmento.
Congestión	Categoría baja, media, alta o bloqueada.



La figura es una visualización del grafo usado por el sistema. Los nodos representan intersecciones y las líneas representan segmentos viales. La aplicación también ofrece una vista interactiva sobre Leaflet y una vista topológica con Cytoscape.

7. Dataset inicial

7.1 Fuente y método de obtención

El dataset principal proviene de OpenStreetMap y fue descargado con OSMnx 2.1.1 usando una red de tipo drive alrededor de la Universidad Sergio Arboleda. Este tipo de red incluye vías transitables por automóvil y conserva su orientación. Los datos se transformaron al modelo interno y se exportaron a JSON, GraphML, CSV y GeoJSON.

Métrica	Valor observado
Nodos	888
Aristas dirigidas	1.741
Aristas con geometría vial	1.127
Distancia promedio por arista	82.39 m
Tiempo estimado promedio por arista	0.099 min
Velocidad promedio	33.74 km/h
Congestión actual	Baja en todas las aristas; valor inicial, no medición en tiempo real

7.2 Estructura de los nodos

Campo	Descripción
id	Identificador único de OpenStreetMap convertido a texto.
nombre	Etiqueta legible del nodo.
latitud / longitud	Coordenadas geográficas WGS84.
tipo	En el dataset actual: intersección.
atributos	Información adicional, como accesibilidad o descripción.

7.3 Estructura de las aristas

Campo	Descripción
id	Identificador único del segmento.
origen / destino	Nodos conectados y sentido permitido.
distancia	Longitud del segmento en metros.
tiempo_estimado	Tiempo calculado a partir de distancia y velocidad.
velocidad_promedio	Valor OSM o valor derivado del tipo de vía.
congestión	Baja, media, alta o bloqueada.
disponible / incidentes	Estado transitable y número de incidentes.
atributos	Tipo, nombre, carriles, sentido único, peaje y geometría.

7.4 Limitaciones del dataset

- La cobertura es una zona de estudio dentro de Chapinero, no toda la localidad.
- Los datos dependen de la calidad y fecha de actualización de OpenStreetMap.
- El tiempo estimado usa velocidades disponibles o inferidas; todavía no incorpora tráfico real.
- La congestión se inicializa como baja y debe sustituirse por datos históricos o simulaciones documentadas.

8. Implementación del grafo

El backend utiliza modelos Pydantic para validar nodos y aristas, y NetworkX para construir la estructura computacional. La capa de repositorio permite cargar, guardar y exportar el grafo. La API expone el grafo completo, sus estadísticas y formatos geográficos para el frontend.

```
G = nx.DiGraph()
for nodo in grafo.nodos.values():
    G.add_node(nodo.id, **nodo.model_dump())
for arista in grafo.aristas:
    if arista.disponible:
        G.add_edge(arista.origen, arista.destino,
                    distancia=arista.distancia,
                    tiempo=arista.tiempo_estimado)
```

El archivo GraphML permite inspeccionar la red en herramientas externas, mientras que JSON y GeoJSON alimentan la API y las visualizaciones.

9. Arquitectura del prototipo

Capa	Tecnología	Responsabilidad
Interfaz	Nuxt / Vue	Seleccionar origen, destino y criterio; mostrar métricas.
Visualización	Leaflet / Cytoscape	Mapa geográfico y vista topológica.
API	FastAPI	Exponer grafo, mapa, estadísticas y ciclo del agente.
Dominio	Pydantic / NetworkX	Validar datos y operar la red dirigida.
Agente	Python	Percibir, decidir, ejecutar y medir el avance.
Datos	OSM / JSON / GraphML / GeoJSON / CSV	Persistencia e intercambio de la red.

Flujo actual del prototipo:

Usuario → selecciona origen y destino → API carga el grafo → agente percibe conexiones → decisión local → acción → actualización del estado → visualización

10. Prototipo del agente

El módulo MotorDecision ejecuta un ciclo de percepción, decisión y acción. Obtiene las acciones posibles desde la ubicación actual, escoge una conexión mediante una regla greedy local, registra la acción y actualiza la medida de desempeño. La API permite ejecutar un paso y consultar el estado del agente.

Acción	Descripción
avanzar	Desplazarse por una arista disponible al siguiente nodo.
esperar	Permanecer en la ubicación cuando la situación lo requiera.
recalcular	Solicitar una nueva decisión desde la ubicación actual.
finalizar	Terminar cuando se alcanza el destino.

La búsqueda completa del botón de la interfaz aún es un placeholder y solo resalta una conexión directa cuando existe. No se presenta como algoritmo final. La integración de BFS, DFS, UCS, búsqueda voraz y A* se realizará en el segundo corte.

11. Pruebas y validaciones disponibles

El repositorio contiene pruebas unitarias para los modelos del grafo, el PEAS y el motor de decisión. La evidencia entregada debe incluir la salida obtenida al ejecutar pytest en el entorno del equipo. En esta revisión documental no se reporta un porcentaje de cobertura como resultado verificado, ya que debe regenerarse con las dependencias instaladas.

Validación	Propósito
Modelos del grafo	Verificar tipos, atributos y restricciones de nodos y aristas.
Modelo PEAS	Comprobar costo, ambiente, percepción y desempeño.
Motor de decisión	Validar estados, acciones y terminación del ciclo.
Integridad del dataset	Confirmar IDs, referencias origen-destino y valores numéricos.
Prueba funcional	Seleccionar dos nodos, consultar la API y observar el mapa.

Comando para producir evidencia reproducible:

```
cd backend
pytest -v --cov=src --cov-report=term-missing
```

12. Cumplimiento de la primera entrega

Entregable de la guía	Estado	Ubicación o evidencia
Documento de formulación	Cumplido	Este informe: problema, alcance y objetivos.
Modelo PEAS	Cumplido	Sección 5 y backend/src/agente/peas.py.
Diagrama de la red	Cumplido	Figura 1 y visualizaciones del frontend.
Dataset inicial	Cumplido	backend/datos/dataset_osm.json y derivados.
Código del grafo	Cumplido	backend/src/grafos y archivos GraphML.

Prototipo del agente	Cumplido con limitaciones	Ciclo local y endpoints del agente.
Pruebas	Parcial	Existen pruebas; falta adjuntar una ejecución actual.
Presentación del avance	Pendiente del equipo	Debe prepararse para la sustentación.

13. Plan de trabajo

Fase	Trabajo siguiente
Cierre del Corte 1	Renombrar textos residuales de TransMilenio en código e interfaz; ejecutar pruebas; añadir capturas.
Corte 2	Implementar BFS, DFS, UCS, voraz y A*; definir heurística; comparar métricas.
Corte 3	Depurar datos históricos; predecir tiempo; clasificar congestión; agrupar zonas.
Ampliación taxis	Agregar ubicación del taxi, recogida y destino; comparar taxis y calcular las dos etapas.

Referencias

OpenStreetMap contributors. (2026). OpenStreetMap. <https://www.openstreetmap.org/>

Repositorio del proyecto. <https://github.com/JAGR1792/Proyecto-IA>